A Systematic Comparison of Execution-Free Verification Formulations for Code Repair

Anonymous ACL submission

Abstract

LLM-based program repair increasingly leverages test-time compute scaling, generating multiple candidate patches and selecting the best via a verifier. Execution-based verification is reliable but expensive; three execution-free alternatives have emerged: per-test binary classification (CLS), execution output generation (CWM), and trajectory-level patch scoring (TRAJ). No prior work compares these formulations under matched conditions.

We present the first systematic comparison across two representative code LLM families (Qwen3, DeepSeek-Coder), two complexity levels, and multiple seeds under $\leq$8B LoRA fine-tuning. At function level, CLS and CWM are statistically equivalent across both families (TOST, 5 seeds): classification is not inherently easier than generation when outputs are short. TRAJ parity is model-dependent and not confirmed after multiple comparison correction. At repository level, a significant interaction ($z = -23.7$, DeepSeek-Coder) confirms this equivalence collapses: the CLS–CWM gap widens to 18–26 pp across 2–3 seeds per model under our $\leq$8B LoRA protocol in variant-level evaluation, driven by format collapse in generation-based simulation. Gap magnitude depends on multiple confounds. Exploratory 14B results (single seed) show the function-level gap narrowing to near-zero (-0.05 pp), suggesting capacity-dependent convergence.

Error complementarity and a failure mode taxonomy motivate complexity-aware formulation routing over uniform selection.

1 Introduction

LLM-based automated program repair systems increasingly rely on test-time compute scaling: generating K candidate patches and selecting the best via a verifier (Snell et al., 2025). Full execution verification is reliable but expensive ($1–3 per candidate at repository scale[1]), driving demand for execution-free alternatives. Three distinct formulations have emerged for this task: **generation-based simulation** (FAIR CodeGen team et al., 2025; Maimon et al., 2026) predicts execution output strings per test; **trajectory-level scoring** (Shum et al., 2025) predicts a binary patch verdict without per-test decomposition; and **per-test classification** (Ni et al., 2023; Chen et al., 2023) predicts a pass/fail token per test. Each makes different tradeoffs between diagnostic granularity, output complexity, and inference cost.

Despite active development along each axis, no prior work provides a systematic comparison holding model capacity, training data, and compute constant. Practitioners choosing a formulation for their verification pipeline must rely on results from different models, different datasets, and different evaluation protocols. Whether one formulation is uniformly preferable, or whether the choice depends on properties of the code being verified, remains an open empirical question. A related question is whether formulation choice matters at all relative to more basic decisions: at what complexity level does fine-tuning become necessary, and does formulation selection add value beyond what fine-tuning alone provides?

We present the first systematic empirical comparison across two representative code LLM families (Qwen3, DeepSeek-Coder), two complexity levels (function-level and repository-level), and multiple seeds. Our contribution is primarily methodological: a matched experimental framework that isolates formulation effects from confounding variables. The analysis also produces actionable outputs: a failure mode taxonomy that characterizes when and why each formulation fails (§3.5), error complementarity findings with practi-

[1] Based on per-instance costs reported by Yang et al. (2024) and Xia et al. (2025).

cal ensemble implications (§3.6), and formulation selection recommendations conditioned on code complexity (§5). The results reveal a three-layer structure. Fine-tuning is necessary (0-shot→fine-tuned gains of 35–36 pp at repo level). After fine-tuning, *formulation choice is moderated by code complexity*: at function level, all formulations achieve comparable accuracy—consistent with the theoretical analysis in §2.2, which predicts that short outputs minimize generation-specific failures. At repository level under our ≤8B LoRA protocol, this equivalence collapses; variant-level analysis reveals three distinct failure modes consistent across model families, though gap magnitude depends on multiple confounds (§3.7). Figure 1 illustrates the three formulations.

We make the following contributions:

- **Complexity-moderated formulation choice.** We establish a three-layer structure: (1) fine-tuning is necessary (35–36 pp gains); (2) at function level, formulations are near-equivalent—CLS and CWM achieve statistical equivalence (TOST p=0.007, Holm-corrected) across both families, while TRAJ equivalence is not confirmed after multiple comparison correction; (3) at repository level under ≤8B LoRA, this equivalence collapses ($z = -23.7$, DeepSeek-Coder), with CWM falling 18–26 pp below CLS, driven by format collapse—four interventions all fail to close the gap. The TOST framework demonstrates strict equivalence rather than mere non-significance (§3.4, §3.5).
- **Error complementarity.** CLS and CWM exhibit partially complementary error patterns at function level (oracle: 90.63%, 95% CI [89.3%, 91.9%]; single-seed); at repo level, oracle routing reaches 95.83% variant accuracy (+14.58 pp; single-seed), the strongest practical argument for exploring formulation-level tradeoffs (§3.6).

2 Method

Let p denote a candidate patch and $T = \{t_1, \ldots, t_n\}$ its test suite, where each t_i has label $y_i \in \{\text{PASS}, \text{FAIL}\}$ from execution. Tests decompose into *fail-to-pass* (T_{f2p}) and *pass-to-pass* (T_{p2p}) subsets; a patch is correct iff $\forall\, t_i$: $y_i = \text{PASS}$. We compare three formulations for predicting correctness without execution.

2.1 Verification Formulations

Let c_p denote the code context (unified diff) of patch p.

CLS (per-test classification). $f(c_p, t_i) \to \hat{y}_i \in \{\text{PASS}, \text{FAIL}\}$ predicts a single binary token per test (cross-entropy loss). Patch verdict: $\hat{y}_p = \bigwedge_i [\hat{y}_i = \text{PASS}]$. Output cost: one token per test.

CWM (output generation). $g(c_p, t_i) \to \hat{o}_i \in \Sigma^*$ generates the execution output string (language modeling loss). Verdict by exact match: $\hat{y}_i = \text{PASS}$ iff $\hat{o}_i = o_i^*$.[2] When the generated output is unparseable (i.e., does not conform to the expected structured format), the prediction defaults to the majority class verdict. Output cost: $\sum_i |\hat{o}_i|$ tokens.

TRAJ (trajectory-level). $h(c_p, T) \to \hat{y}_p \in \{\text{RESOLVED}, \text{UNRESOLVED}\}$ predicts a single patch verdict from all tests and provides no per-test signal. TRAJ serves as a granularity control: CLS–TRAJ differences isolate the value of per-test decomposition independent of the classification formulation itself.

2.2 Formulation-Specific Failure Modes

Each formulation has characteristic failure modes that differ in their sensitivity to code complexity:

Generation failure modes (CWM). The open-ended output space introduces three failure modes: (1) *token-budget exhaustion*, where $|o_i^*| > B$ guarantees exact-match failure due to truncation; (2) *string-value prediction errors*, where semantically equivalent outputs differing in surface form (floating-point precision, dictionary ordering) produce false negatives; and (3) *format compliance collapse*, where the model fails to produce structured predictions entirely at high input complexity, even when the generation budget is sufficient. Under optimistic independence, a token sequence of length L with per-token accuracy q has correct-sequence probability q^L; long outputs are therefore exponentially harder to predict exactly. We adopt exact string matching following the original CWM evaluation protocol (FAIR CodeGen team et al., 2025). Unparseable predictions default to the majority-class label (PASS).

[2] Exact string matching is conservative, consistent with FAIR CodeGen team et al. (2025). Semantic equivalences are not counted.

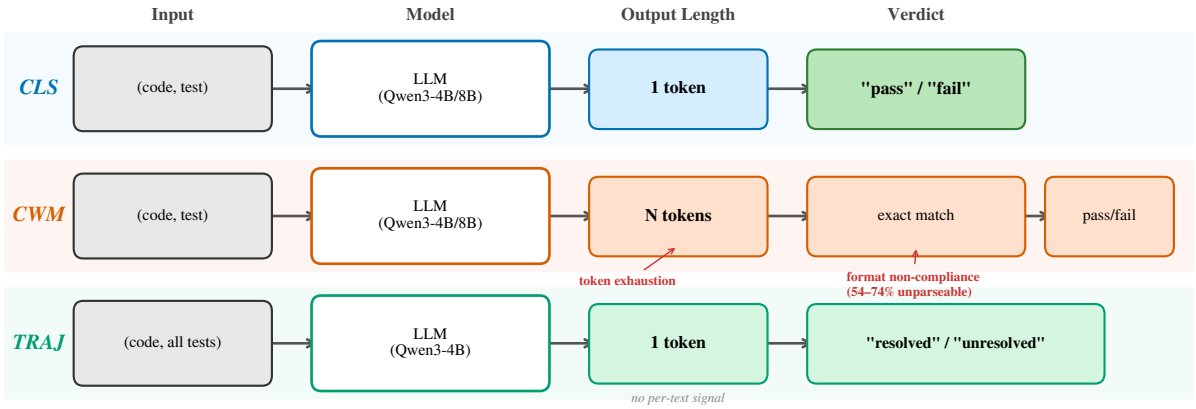


Figure 1: Three verification formulations. **Left:** CLS predicts binary pass/fail per test (1 output token). **Center:** CWM generates execution output, derives verdict by exact match ($O(L)$ tokens). **Right:** Trajectory-level scoring predicts patch-level resolved/unresolved (1 token, no per-test signal). Each formulation makes different tradeoffs between output complexity, diagnostic granularity, and inference cost.

Classification failure modes (CLS). The single-token output space eliminates format and truncation failures but constrains the model to encode its reasoning entirely within internal representations, without the intermediate reasoning trace that generation provides. CLS errors manifest as false positives or false negatives without format failures; at function level, a mild FP bias (56.5%) emerges, with the error direction depending on class distribution and fine-tuning dynamics.

Trajectory-level failure modes (TRAJ). By collapsing all per-test information into a single binary verdict, TRAJ discards diagnostic granularity. When a patch fails multiple tests, the trajectory-level formulation cannot identify *which* tests fail. This limits downstream utility (e.g., targeted repair) but avoids per-test aggregation errors.

Complexity dependence. These failure modes suggest a complexity-dependent interaction: when execution outputs are short and structured (function-level code), generation modes (1)–(2) are rarely triggered, and all formulations should perform comparably. As output length and structural complexity increase (repository-level code), generation-specific failures may compound, creating divergence. Whether this divergence reflects a fundamental property of the generation formulation or is contingent on model/tokenizer properties is an empirical question we investigate in §3.

2.3 Oracle Accuracy-Utility Framework

For a buggy patch with $k = |T_{\text{f2p}}|$ fail-to-pass tests, a per-test verifier with recall r detects the bug if it identifies at least one failing test: $P_{\text{detect}}(k) = 1 - (1 - r)^{k_{\text{eff}}}$, where $k_{\text{eff}} = k/(1 + (k - 1)\rho)$ accounts for within-patch correlation via ICC ρ (Shrout and Fleiss, 1979). The per-test advantage over a trajectory-level baseline (a_h) is $\Delta(k) = P_{\text{detect}}(k) - a_h$, with population expectation $\mathbb{E}[\Delta] = \sum_k P(|T_{\text{f2p}}| = k) \cdot \Delta(k)$. This predicts that per-test advantage concentrates in multi-F2P-test instances and vanishes at k=1, validated in §3 and Appendix A.

3 Experiments

We systematically compare CLS, CWM, and TRAJ across two representative code LLM families, two complexity levels, and multiple seeds to characterize the formulation landscape for neural code verification.

3.1 Experimental Setup

Datasets. *Function-level*: mutated variants from HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) (five mutation types), labels verified by execution. 19,076 samples (train 15,290 / val 1,896 / test 1,890; pass:fail ≈ 52:48, majority baseline 51.75%). *Repo-level*: per-test instances from SWE-bench Verified (Chowdhury et al., 2024) mutants. ∼70K training samples; val (n=9,335) and test (n=9,784) have zero problem overlap (majority baseline 72.96%). Val-based checkpoint selection follows standard prac-

tice (Jimenez et al., 2024) where validation loss is computable; runs using truncate preprocessing produce NaN validation loss because repo-level execution traces often exceed the 2,048-token sequence length limit; after truncation, many samples have all target tokens removed (labels entirely -100), producing degenerate loss—a formulation-inherent weakness that classification and trajectory formulations avoid by design. NaN loss defaults to the final epoch checkpoint (affects DeepSeek CLS s123, all CWM runs, and Qwen3 TRAJ runs; see Appendix E).

Models and training. We fine-tune Qwen3-4B and Qwen3-8B (Yang et al., 2025a) with Low-Rank Adaptation (LoRA) (Hu et al., 2022) (rank 16, 3 epochs) across five seeds. Seeds 42/123/456 use bfloat16; seeds 789/0 use NormalFloat 4-bit (NF4) quantization (Dettmers et al., 2023) to test robustness to precision reduction (function-level accuracy remains within seed noise across both precisions; see footnote in §3.4). As a cross-model consistency check, we additionally fine-tune DeepSeek-Coder-6.7B (Guo et al., 2024) with identical LoRA configuration (2 seeds each for CLS and CWM). Our CWM baseline is a LoRA fine-tuned variant for matched comparison, not a reproduction of the 32B Code World Model (FAIR CodeGen team et al., 2025). Repo-level models use val-selected checkpoints where available; runs with NaN validation loss use the final epoch checkpoint (Appendix E). Zero-shot baselines use Qwen3-8B and Qwen3-32B (NF4) as LLM-as-judge. Full hyperparameter details appear in Appendix E.

Statistics. Confidence intervals use clustered bootstrap (Davison and Hinkley, 1997) (114 problem clusters, 10K resamples; we resample entire problems, not individual tests, to preserve within-problem correlations); per-test vs. trajectory-level equivalence uses Two One-Sided Tests (TOST) (Schuirmann, 1987) with ± 2 percentage point (pp) margin. Holm-Bonferroni correction (Holm, 1979) controls family-wise error rate across the five primary comparisons (Table 15). Intraclass correlation coefficient (ICC) (Shrout and Fleiss, 1979) quantifies within-problem correlation. Variant-level effect sizes and statistical power analysis are reported in Appendix K. Oracle analysis uses SWE-bench Verified (500 instances).

Table 1: Function-level per-test accuracy on HumanEval+MBPP (n=1,890). Mean $\pm$ std over seeds. Seeds 42/123/456: bfloat16; 789/0: NF4. DeepSeek: n=2 seeds; std reported for descriptive comparison, not inferential. TRAJ evaluated on 1,032-sample subset (447 matched variants). All prompting baselines use NF4 4-bit quantization for fair cross-scale comparison.

Method	Model	Seeds	Accuracy (%)
Qwen3 (primary)			
CLS (fine-tuned)	4B	5	**88.43 ± 0.78**
CWM (fine-tuned)	4B	5	87.19 ± 0.37
TRAJ (fine-tuned)	4B	5	86.30 ± 0.97
DeepSeek-Coder (cross-model consistency check)			
CLS (fine-tuned)	6.7B	2	**86.85 ± 0.11**
CWM (fine-tuned)	6.7B	2	86.83 ± 1.49
TRAJ (fine-tuned)	6.7B	2	84.16 ± 1.30
Prompting baselines			
0-shot	8B (Qwen3)	1	60.53
0-shot	14B (Qwen3)	1	66.24
0-shot	6.7B (DeepSeek)	1	63.00
ZS LLM-as-judge	8B (Qwen3)	1	64.40
ZS LLM-as-judge	6.7B (DeepSeek)	1	56.56
ZS LLM-as-judge	32B (NF4)	1	71.69
3-shot ICL	8B (Qwen3)	1	60.42
3-shot ICL	14B (Qwen3)	1	73.12
3-shot ICL	6.7B (DeepSeek)	1	59.95
Majority (pass)	—	—	51.75

3.2 Function-Level Results

Table 1 reports function-level results across formulations and model families.

All three formulations achieve comparable accuracy on Qwen3-4B: CLS 88.43% ($\pm$0.78), CWM 87.19% ($\pm$0.37), TRAJ 86.30% ($\pm$0.97). DeepSeek-Coder-6.7B shows: CLS 86.85%, CWM 86.83%, TRAJ 84.16%. The CLS–CWM gap (0.02–1.24 pp) is within seed noise in both families. However, the CLS–TRAJ gap is model-dependent: $<$1 pp on Qwen3 but 2.69 pp on DeepSeek-Coder, exceeding the $\pm$2 pp equivalence margin (§3.4). CLS and CWM are robustly equivalent; TRAJ parity is model-contingent. Zero-shot baselines (Tables 1 and 3) establish that repo-level verification requires fine-tuning (0-shot$\rightarrow$fine-tuned gains of 35–36 pp) and that formulation choice matters only at repo level (CLS outperforms CWM by 18–26 pp).

3.3 Scaling Behavior

Beyond overall accuracy, we examine whether formulation ranking is stable across model sizes. Table 2 compares all three formulations at 4B, 8B, and 14B. The 4B and 8B rows use seeds 42 and 123; the 14B row uses seed 42 (single-seed, exploratory). All rows use bfloat16 inference.

Table 2: Model-size scaling: accuracy (%) at 4B, 8B, and 14B (Qwen3). Seeds 42/123 (4B/8B), seed 42 (14B, exploratory). All bfloat16 inference. Δ: change from previous size. CWM comparison is exploratory (n=2 seeds at 4B/8B, n=1 at 14B).

Method	Size	s42	s123	Mean	Δ
CLS	4B	87.83	89.52	**88.68**	—
	8B	**90.48**	89.40	**89.94**	+1.27
	14B	89.21	—	89.21†	−0.73
CWM	4B	87.51	87.20	87.36	—
	8B	87.67	86.98	87.33	−0.03
	14B	89.26	—	89.26†	+1.93
TRAJ	4B	85.66	85.47	85.57	—
	8B	87.60	87.79	87.70	+2.13
	14B	—	—	—	—

†Single seed (42); exploratory.

CLS and TRAJ both scale positively from 4B to 8B (+1.27 pp and +2.13 pp respectively), while CWM remains flat (−0.03 pp). At 8B, the ranking is CLS (89.94) > TRAJ (87.70) > CWM (87.33). The CLS–CWM gap widens from 1.32 pp at 4B to 2.62 pp at 8B (Figure 2); clustered bootstrap on the 3-seed superset (42, 123, 456 for CLS/CWM) yields $p = 0.0066$ (95% CI $[1.80, 2.71]$), which confirms the widening is not a sampling artifact.

At 14B (single seed; exploratory), CLS reaches 89.21% (−0.73 pp) while CWM reaches 89.26% (+1.93 pp), closing the gap from 2.62 pp at 8B to −0.05 pp—effectively zero. CWM's delayed scaling onset (flat 4B→8B, then +1.93 pp at 14B) suggests a capacity threshold for output formatting, rather than a fundamental formulation limitation. Under matched precision (bfloat16 or NF4), the 14B gap remains <0.3 pp, confirming convergence is robust to quantization. These single-seed results require multi-seed confirmation.

Baseline scaling. Under NF4 quantization, prompting baselines show a scale×few-shot interaction: Qwen3-8B gains nothing from demonstrations ($\Delta = -0.11$ pp), while 14B gains +6.88 pp—consistent with emergent ICL ability (Wei et al., 2022). The best prompting result (14B 3-shot, 73.12%) remains 16.82 pp below fine-tuned CLS at 8B, confirming fine-tuning is necessary at function level. At function level, format compliance is near-perfect for all formulations; at repo level, output complexity increases substantially and the generation bottleneck manifests as format collapse (§3.5). TRAJ shows no scaling benefit at repo level (72.92% at both 4B and 8B); only CLS scales positively across both complexity levels.

3.4 Function-Level Equivalence

We test pairwise equivalence on 523 variant-level instances across five seeds.[3] Our ±2 pp SESOI threshold (Lakens, 2017; Wellek, 2010) is set a priori: at 85–88% accuracy, 2 pp corresponds to ∼2–3 problems out of 114, below practitioner-relevant differences, and is consistent with between-seed variation (0.37–0.97 pp std). Within-problem predictions are correlated (ICC = 0.191, DEFF = 3.97 (Moulton, 1990)); all tests use clustered bootstrap.

CLS–CWM equivalence. The pooled gap is 1.24 pp (88.43%–87.19%); TOST confirms equivalence at $p = 0.007$ (±2 pp margin), surviving Holm-Bonferroni correction (k=5, adjusted p=0.028; Table 15). Equivalence holds at margins as tight as ±1.52 pp (Table 13). On DeepSeek-Coder, the CLS–CWM gap of 0.02 pp is consistent with equivalence but underpowered (2 seeds).

CLS–TRAJ equivalence. The pooled gap is −0.45 pp (95% CI [−1.71, +0.84]); TOST yields $p = 0.025$ (±2 pp), nominally significant but not surviving Holm-Bonferroni correction (k=5, adjusted p=0.075; Table 15). The DeepSeek CLS–TRAJ gap (2.69 pp) exceeds the ±2 pp margin; TRAJ parity is model-dependent. Per-test classification provides diagnostic information (which tests fail) at no cost to variant-level accuracy, paralleling process supervision (Lightman et al., 2023).

Statistical power. With $n_{\text{eff}} \approx 86$ after clustering, TOST power is ∼0.3 (Cohen, 1988). The CLS–CWM equivalence (p=0.007) is robust; CLS–TRAJ non-confirmation may reflect insufficient power. A Bayesian ROPE analysis corroborates (P(ROPE)=0.975 for CLS–CWM; Appendix J).

3.5 Repo-Level: Complexity-Dependent Divergence

Table 3 compares formulations at repository-level complexity. Variant-level comparisons (n=48) are exploratory due to limited seed coverage; the per-test interaction effect is statistically confirmed (see

[3]Seeds 789/0 use NF4 inference; the remaining three use bfloat16. Restricting to bfloat16 only: CLS–CWM remains significant (p<0.05); CLS–TRAJ yields p=0.128.

Table 3: Repo-level cross-formulation comparison on SWE-bench Verified test set. Qwen3-8B CLS/TRAJ: mean ± std over 3 seeds; DeepSeek CLS: 2 seeds[‡]; DeepSeek TRAJ: 3 seeds; Qwen3-4B: seed 42. CWM: 3 seeds (42/123/456).

Model	CLS (%)	CLS Var. (%)	CWM (%)	TRAJ Var. (%)[◇]
Qwen3-4B	83.63	64.58	58.04	72.92
Qwen3-8B	**84.83 ± 0.67**	72.92 / 81.25[†]	66.48 ± 1.15	72.91 ± 3.61
DeepSeek-6.7B	84.38 ± 2.50[‡]	81.25[†]	58.30 ± 2.82	77.08 ± 3.61
0-shot (Qwen3-8B)	49.90	—	—	—
0-shot (DeepSeek)	48.54	—	—	—
Majority	72.96	54.17	72.96	54.17

[†]Individual-seed variant accuracy. [‡]One seed excluded: truncate preprocessing→NaN val loss, decision collapse (Table 4); mean over two matched seeds. [◇]Var. columns: variant-level accuracy (n=48); CLS aggregated via all-pass rule; TRAJ by design. Per-test: n=9,784. CWM unparseable: 54% (4B); default to pass; parseable-only 38–43%, below majority. Eval seq_length differs across models (Appendix O). CWM uses epoch-3 checkpoint (val loss undefined); see text for checkpoint sensitivity analysis.

below). Checkpoints are val-selected where validation loss is computable (§3.1); the test set has zero problem overlap with validation.

Few-shot prompting (3-shot ICL) collapses to an always-predict-unresolved baseline (variant accuracy 45.83%, below majority 54.17%; Appendix B), establishing that execution-free verification requires task-specific fine-tuning regardless of formulation. The fine-tuning requirement (35–36 pp gains, Table 3) is the first layer of our three-layer structure.

Formulation divergence. Under our 4–8B LoRA protocol, CLS and CWM diverge on Qwen3: the gap is 25.59 pp at 4B (CLS 83.63% vs. CWM 58.04%, seed 42; Table 3) and 18.35 pp at 8B (84.83% vs. 66.48%, 3-seed means). CWM falls below majority baseline at both scales. These gaps confirm complexity-dependent formulation divergence, though limited seed coverage and multiple confounds (§3.7) limit definitive causal attribution.

Cross-model consistency. DeepSeek-Coder shows the same direction (CLS > CWM) across all seeds, with gap magnitude varying across families (20 pp Qwen3 vs. 23–28 pp DeepSeek), primarily reflecting checkpoint selection sensitivity; under matched epoch-3 checkpoints, the cross-model gap difference narrows to ∼2 pp (Appendix O). Across all six CWM repo-level seeds, every seed exhibits format collapse (>50% unparseable) and parseable-only accuracy below majority baseline, ruling out seed-specific artifacts.

Table 4: Repo-level failure mode taxonomy. Variant-level majority baseline: 54.17%.

Failure Mode	Representative	Fail Recall	Variant Acc
Format collapse	CWM (all)	N/A	48–52%
Decision collapse	CLS (DeepSeek, s123)	∼14%	56%
Aggr. penalty	CLS (Qwen3-8B)	51–57%	73–81%
Aggr. penalty	CLS (DeepSeek, s42)	38–51%	79–81%

Failure mode taxonomy. Variant-level analysis (n=48) reveals three failure modes (Table 4): format collapse (CWM, 51–74% unparseable), decision collapse (CLS under truncate preprocessing, fail recall ∼14%), and aggregation penalty (CLS under drop preprocessing, fail recall 38–57%). Decision collapse is configuration-dependent, not model-inherent. TRAJ avoids aggregation penalty by predicting variant outcomes directly (72.91–77.08%). Downsampling CLS to match TRAJ's data volume still yields 80.74% (+7.82 pp above TRAJ; Appendix B).

Default assignment artifact. The apparent CWM accuracy increase from 4B to 8B is an artifact of rising unparseable rate (53.7%→73.8%), inflating accuracy via majority-class default while parseable-only accuracy decreases (42.88%→38.52%); cross-seed evidence confirms the pattern (Appendix N).

Complexity as moderator. The CLS–CWM gap widens from 1–3 pp (function) to 18–26 pp (repo) under our protocol; a matched-context ablation confirms input disparity explains <3% of this gap (Appendix B).

Formulation×complexity interaction. A mixed-effects LPM (n=56,480, 139 clusters, DeepSeek-Coder) confirms: CWM effect at function level is negligible ($\hat{\beta}=-0.002$, p=0.805), but the formulation×complexity interaction is highly significant ($z=-23.7$, $p<0.001$). A cross-model LPM confirms formulation dominates ($\hat{\beta}=-0.22$ vs. model $\hat{\beta}=+0.011$; Appendix M).

Additional repo-level limitations appear in Appendix P.

3.6 Analysis

Aggregation penalty. Under the all-pass rule, a single false negative flips the variant verdict; the penalty ranges from 1.36 pp to 19 pp depending on fail recall and test count (Appendix Table 10). TRAJ avoids this by predicting variant outcomes directly (72.91–77.08%). The penalty concen-

trates at high test counts: negligible at function level ($\bar{T} \approx 2.3$) but substantial at repo level ($\bar{T} \gg 10$; Appendix I(a)).

Why does generation degrade at repo level? Output complexity increases nonlinearly: function-level outputs average ∼5–15 tokens; repo-level outputs average ∼477 tokens, causing format collapse (53–74% unparseable). Even parseable outputs achieve only 38–43%, below majority; forced parsing confirms the bottleneck is semantic, not syntactic (Appendix I(c)). Four interventions (LoRA rank 32, voting, relaxed matching, scaling) all fail (Appendix I(g)). Checkpoint selection does not explain the gap: matched epoch-3 comparisons yield 20–23 pp gaps, and CLS varies <1 pp across epochs (Appendix O). This extends formulation-dependent tradeoffs (Raffel et al., 2020) to a regime where format compliance dominates.

CLS achieves 217× higher throughput than CWM (1 output token vs. avg 477; median latency: 48ms vs. 16.3s per inference; Appendix I(i)), consistent across both model families.

Error complementarity. CLS and CWM exhibit partially complementary errors. An oracle combination achieves 90.63% (+2.80 pp; 95% CI [89.3%, 91.9%]; single-seed) at function level and 95.83% (+14.58 pp; n=48) at repo level. Simple routing heuristics capture <0.4 pp of the oracle gap; the complementarity is instance-specific, suggesting a learned router is needed (Appendix A).

3.7 Threats to Validity

We identify five potential confounds and assess each in Appendix O: checkpoint selection (<1 pp variation in ablation; NaN validation loss forces final-epoch default in all CWM runs and DeepSeek CLS s123), evaluation sequence length (∼1.5 pp), preprocessing (∼8 pp cross-configuration variance), TRAJ training regime (downsample confirms formulation advantage), and CWM scale/pretraining (primary limitation; all repo-level CWM conclusions conditional on the ≤8B LoRA regime). The 18–26 pp gap exceeds any single assessed confound, but correlated confounds and limited seeds mean definitive causal attribution requires multi-seed replication. Holm-Bonferroni correction (k=5) confirms the two core claims survive (adjusted p=0.015 and p=0.028; Table 15): repo-level CLS–CWM divergence and function-level CLS–CWM equivalence.

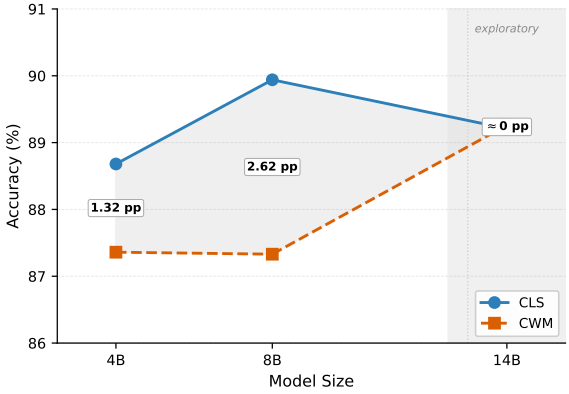


Figure 2: Function-level accuracy scaling (4B–14B). CLS (solid) and CWM (dashed) accuracy with shaded gap region. The CLS–CWM gap follows an inverted-U trajectory: 1.32 pp at 4B, widening to 2.62 pp at 8B, then converging to near-zero (−0.05 pp) at 14B. 14B results are single-seed (exploratory).

4 Related Work

Execution Simulation and Verification. Execution trace generation improves code reasoning (Nye et al., 2021), with extensions to output prediction (FAIR CodeGen team et al., 2025; Maimon et al., 2026; Liu et al., 2024, 2023; Lee and Huang, 2026), failure analysis (Abdollahi et al., 2025), and error detection (Li et al., 2024). CRUXEval (Gu et al., 2024) benchmarks code execution prediction at function level; RepoExec (Nam et al., 2025) evaluates repo-level code generation with execution-based correctness checks. Our work extends execution prediction to repo-level repair verification. CodeScaler (Zhu et al., 2026) trains execution-free reward models for both RL and test-time scaling. Verification formulations span trajectory-level reward (Shum et al., 2025), process-level feedback (Li et al., 2025b; Xu et al., 2025), per-test classification (Ni et al., 2023; Chen et al., 2023), learned metrics (Zhuo, 2024; Dong et al., 2024; Zhou et al., 2023), and RL with execution-based rewards (Wang et al., 2026; Tang et al., 2026). Aletheia (Venkatkrishna et al., 2026) compares training recipes; we compare *formulations*. Test-time scaling (Snell et al., 2025; Li et al., 2025a; Chen et al., 2025) and self-consistency (Wang et al., 2023) demonstrate verification value; none compare formulations at matched capacity.

Process vs. Outcome Supervision. PRMs outperform ORMs for math reasoning (Lightman et al., 2023; Uesato et al., 2022), with automated

annotation (Wang et al., 2024; Luo et al., 2024) and code-specific extensions (Zhang et al., 2025; Ye et al., 2025; Zhang et al., 2026; Jiang et al., 2025). GenRM (Hosseini et al., 2025) and GenPRM (Zhao et al., 2026) show that generative verifiers outperform discriminative ones for math reasoning; our results reveal a domain difference—in code verification, the generative formulation (CWM) does not outperform the discriminative one (CLS), suggesting that the generation advantage may be domain-specific. CodeRM (Ma et al., 2025) demonstrates that test-level reward granularity improves code generation quality, corroborating our finding that per-test classification provides diagnostic value. In reward modeling terminology, CLS corresponds to process-level (PRM-like) verification while TRAJ provides outcome-level (ORM-like) feedback; CWM serves as a generative verifier (Hosseini et al., 2025). Our CLS provides per-test (process) supervision while TRAJ provides outcome-level; the PRM literature compares supervision granularity given a fixed output format, while we additionally vary the output format (CWM generation vs. CLS classification).

Repair, Mutation Testing, and Task Framing. CodeRL (Le et al., 2022), AlphaCode (Li et al., 2022; AlphaCode Team, 2023), and Reflexion (Shinn et al., 2023) use execution feedback for repair. SWE-bench (Jimenez et al., 2024; Chowdhury et al., 2024) provides benchmarks; SWE-agent (Yang et al., 2024) and Agentless (Xia et al., 2025) demonstrate pipelines where verification is a bottleneck (Yang et al., 2025b). Predictive mutation testing (Tufano et al., 2019; Kim et al., 2022; Wang et al., 2025; Jain et al., 2023) shares our per-test structure. Raffel et al. (2020) showed that task framing affects performance across NLP tasks; we extend this to code verification, finding that complexity moderates the classification–generation tradeoff.

5 Conclusion

We presented the first systematic comparison of three execution-free verification formulations (CLS, CWM, TRAJ). At function level, CLS and CWM are statistically equivalent (TOST p=0.007, 5 seeds); TRAJ equivalence is not confirmed after multiple comparison correction. Per-test classification achieves 217× higher throughput while providing diagnostic signal. Complementary errors (oracle: 90.63%/95.83%; single-seed) motivate complexity-aware routing. At repository level under ≤8B LoRA, divergence reaches 18–26 pp ($z = -23.7$, 2–3 seeds per model), with CWM falling below majority baseline—consistent across families but dependent on multiple confounds (§3.7). Three failure modes (format collapse, decision collapse, aggregation penalty) are consistent across families. CLS is recommended primarily for its throughput advantage and diagnostic capability; the accuracy advantage over TRAJ is directional but underpowered (n=48, MDE≈25 pp; Appendix K). Primary statistical findings are established under ≤8B LoRA fine-tuning. Exploratory 14B results show the CLS–CWM gap narrowing to near-zero (−0.05 pp; single seed), providing preliminary evidence that the generation bottleneck is capacity-dependent rather than formulation-inherent.

Three directions follow: (1) oracle gains motivate complexity-aware routing using the failure mode taxonomy as routing features; (2) the semantic generation bottleneck (four interventions fail; Appendix I(g)) suggests capacity scaling or execution trace pretraining (FAIR CodeGen team et al., 2025) is needed—testing at 32B+ would resolve the primary open question; (3) larger-scale evaluation of per-test diagnostics for downstream repair (n=44 in our pilot) would test whether CLS's throughput advantage extends to repair pipelines.

Limitations

Primary findings use ≤8B LoRA fine-tuning with multi-seed replication; 14B results are single-seed and serve as directional evidence only. The generation bottleneck driving the repo-level CLS–CWM gap may diminish at 32B+ scale or with execution-trace pretraining (FAIR CodeGen team et al., 2025), and we cannot confirm whether the formulation ranking generalizes beyond this regime. All evaluation uses Python-only benchmarks (HumanEval, MBPP, SWE-bench Verified); languages with different test output conventions or error formats may exhibit different format compliance rates for generation-based formulations. Training data is constructed via synthetic mutation operators (Andrews et al., 2005; Just et al., 2014) rather than real developer patches; while mutation adequacy research supports relative ranking validity, absolute accuracy levels and the synthetic-to-

real transfer gap remain untested (Appendix P).

References

Mohammad Abdollahi, Khandaker Rifah Tasnia, Soumit Kanti Saha, Jinqiu Yang, Song Wang, and Hadi Hemmati. 2025. Demystifying errors in LLM reasoning traces: An empirical study of code execution simulation. *arXiv preprint arXiv:2512.00215*.

AlphaCode Team. 2023. AlphaCode 2 technical report. Technical report, Google DeepMind.

James H Andrews, Lionel C Briand, and Yvan Labiche. 2005. Is mutation an adequate substitute for real faults? In *ICSE*, pages 313–322.

Joshua D. Angrist and Jörn-Steffen Pischke. 2009. *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press.

Jacob Austin, Augustus Odena, Maxwell Nye, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

A. Colin Cameron, Jonah B. Gelbach, and Douglas L. Miller. 2008. Bootstrap-based improvements for inference with clustered errors. *Review of Economics and Statistics*, 90(3):414–427.

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT: Code generation with generated tests. In *ICLR*.

Hao Mark Chen, Guanxi Lu, Yasuyuki Okoshi, Zhiwen Mo, Masato Motomura, and Hongxiang Fan. 2025. Rethinking optimal verification granularity for compute-efficient test-time scaling. In *NeurIPS*.

Mark Chen, Jerry Tworek, Heewoo Jun, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. 2024. Introducing SWE-bench verified.

Jacob Cohen. 1988. *Statistical Power Analysis for the Behavioral Sciences*, 2nd edition. Lawrence Erlbaum Associates.

Anthony C Davison and David V Hinkley. 1997. *Bootstrap Methods and their Application*. Cambridge University Press.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *NeurIPS*.

Yihong Dong, Jiazheng Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2024. CodeScore: Evaluating code generation by learning code execution. *ACM Transactions on Software Engineering and Methodology*, 33(8).

FAIR CodeGen team, Jade Copet, Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, David Zhang, Kunhao Zheng, and 1 others. 2025. CWM: An open-weights LLM for research on code generation with world models. *arXiv preprint arXiv:2510.02387*.

Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. 2024. CRUXEval: A benchmark for code reasoning, understanding and execution. In *Proceedings of the 41st International Conference on Machine Learning*, pages 16568–16621.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y.K. Li, Fuli Luo, Yun Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*.

John M. Hoenig and Dennis M. Heisey. 2001. The abuse of power: The pervasive fallacy of power calculations for data analysis. *The American Statistician*, 55(1):19–24.

Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Arian Hosseini, Xingdi Yuan, Nikolay Malkin, Aaron Courville, Alessandro Sordoni, and Rishabh Agarwal. 2025. Generative verifiers: Reward modeling as next-token prediction. In *ICLR*.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *ICLR*.

Kush Jain, Ristimoy Barik, and Claire Le Goues. 2023. Contextual predictive mutation testing. In *ESEC/FSE*.

Xue Jiang, Yihong Dong, Mengyang Liu, Hongyi Deng, Tian Wang, Yongding Tao, Rongyu Cao, Binhua Li, Zhi Jin, Wenpin Jiao, Fei Huang, Yongbin Li, and Ge Li. 2025. CodeRL+: Improving code generation via reinforcement with execution semantics alignment. *arXiv preprint arXiv:2510.18471*.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*.

René Just, Darioush Jalali, Laura Inozemtseva, Michael D Ernst, Reid Holmes, and Gordon Fraser. 2014. Are mutants a valid substitute for real faults in software testing? In *FSE*, pages 654–665.

Jinhan Kim, Juyoung Ju, Robert Tao, Nhat-Tan Hu, and Shin Yoo Juyeon Baik. 2022. Predictive mutation testing via natural language channel. *IEEE TSE*.

John K Kruschke. 2013. Bayesian estimation supersedes the t test. *Journal of Experimental Psychology: General*, 142(2):573–603.

John K Kruschke. 2018. Rejecting or accepting parameter values in Bayesian estimation. *Advances in Methods and Practices in Psychological Science*, 1(2):270–280.

Daniël Lakens. 2017. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4):355–362.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C.H. Hoi. 2022. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *NeurIPS*.

Woojin Lee and Jin-Xia Huang. 2026. SolidCoder: Bridging the mental-reality gap in LLM code generation through concrete execution. *arXiv preprint arXiv:2604.19825*.

Dacheng Li, Shiyi Cao, Chengkun Cao, Xiuyu Li, Shangyin Tan, Kurt Keutzer, Jiarong Xing, Joseph E. Gonzalez, and Ion Stoica. 2025a. S*: Test time scaling for code generation. In *Findings of EMNLP*.

Qingyao Li, Xinyi Dai, Xiangyang Li, Weinan Zhang, Yasheng Wang, Ruiming Tang, and Yong Yu. 2025b. CodePRM: Execution feedback-enhanced process reward model for code generation. In *Findings of ACL*, pages 8169–8182.

Shou Li, Andrey Kan, Laurent Callot, Bhavana Bhasker, Muhammad Shihab Rashid, and Timothy B Esler. 2024. REDO: Execution-free runtime error detection for coding agents. *arXiv preprint arXiv:2410.09117*.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, and 1 others. 2022. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. *arXiv preprint arXiv:2305.20050*.

Changshu Liu, Shizhuo Dylan Zhang, Ali Reza Ibrahimzada, and Reyhaneh Jabbarvand. 2024. CodeMind: A framework to challenge large language models for code reasoning. *arXiv preprint arXiv:2402.09664*.

Chenxiao Liu, Shuai Lu, Weizhu Chen, Daxin Jiang, Alexey Svyatkovskiy, Shengyu Fu, Neel Sundaresan, and Nan Duan. 2023. Code execution with pretrained language models. In *Findings of the Association for Computational Linguistics: ACL 2023*.

Liangchen Luo, Yinxiao Liu, Rosanne Liu, Samrat Phatale, Harsh Lara, Yunxuan Li, Lei Shu, Yun Zhu, Lei Meng, Jiao Sun, and Abhinav Rastogi. 2024. Improve mathematical reasoning in language models by automated process supervision. *arXiv preprint arXiv:2406.06592*.

Zeyao Ma, Xiaokang Zhang, Jing Zhang, Jifan Yu, and Jie Tang. 2025. Dynamic scaling of unit tests for code reward modeling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 6917–6935.

Gallil Maimon, Ori Yoran, Felix Kreuk, Michael Hassid, Gal Cohen, Pierre Chambon, and Yossi Adi. 2026. Self-execution simulation improves coding models. *arXiv preprint arXiv:2604.03253*.

Brent R. Moulton. 1990. An illustration of a pitfall in estimating the effects of aggregate variables on micro units. *The Review of Economics and Statistics*, 72(2):334–338.

Hai Nam, Thang Dau, Nhat Bui, and Nghi D. Q. Nguyen. 2025. REPOEXEC: Evaluate code generation with a repository-level executable benchmark. In *NAACL*.

Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida I. Wang, and Xi Victoria Lin. 2023. LEVER: Learning to verify language-to-code generation with execution. In *ICML*.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Biber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. 2021. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21(140):1–67.

Donald J Schuirmann. 1987. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*.

Patrick E Shrout and Joseph L Fleiss. 1979. Intraclass correlations: Uses in assessing rater reliability. *Psychological Bulletin*, 86(2):420–428.

KaShun Shum, Binyuan Hui, Jiawei Chen, Lei Zhang, Jiaxi Yang, Yuzhen Huang, Junyang Lin, and Junxian He. 2025. SWE-RM: Execution-free feedback for software engineering agents. *arXiv preprint arXiv:2512.21919*.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2025. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *ICLR*. Oral.

Lingxiao Tang, He Ye, Zhaoyang Chu, Muyang Ye, Zhongxin Liu, Xiaoxue Ren, and Lingfeng Bao. 2026. ExecVerify: White-box RL with verifiable stepwise rewards for code execution reasoning. *arXiv preprint arXiv:2603.11226*.

Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. Learning how to mutate source code from bug-fixes. In *ISSTA*.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.

Vatsal Venkatkrishna, Indraneil Paul, and Iryna Gurevych. 2026. Aletheia: What makes RLVR for code verifiers tick? *arXiv preprint arXiv:2601.12186*.

Bo Wang, Mingda Chen, Ming Deng, Youfang Lin, Mark Harman, Mike Papadakis, and Jie M. Zhang. 2025. A comprehensive study on large language models for mutation testing. *ACM Transactions on Software Engineering and Methodology*.

Longwen Wang, Xuan'er Wu, Xiaohui Hu, Yirui Liu, Yuankai Fan, Kaidong Yu, Qizhen Weng, Wei Xi, and Xuelong Li. 2026. VeRPO: Verifiable dense reward policy optimization for code generation. *arXiv preprint arXiv:2601.03525*.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *ICLR*.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent abilities of large language models. In *Transactions on Machine Learning Research (TMLR)*.

Stefan Wellek. 2010. *Testing Statistical Hypotheses of Equivalence and Noninferiority*, 2nd edition. Chapman and Hall/CRC.

Brandon T. Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Agentless: Demystifying LLM-based software engineering agents. In *FSE*. Distinguished Paper.

Junjielong Xu, Boyin Tan, Xiaoyuan Liu, Chao Peng, Pengfei Gao, and Pinjia He. 2025. Scalable supervising software agents with patch reasoner. *arXiv preprint arXiv:2510.22775*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, and 1 others. 2025a. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawendé F. Bissyandé, Yang Liu, and Haoye Tian. 2025b. A survey of LLM-based automated program repair: Taxonomies, design paradigms, and applications. *arXiv preprint arXiv:2506.23749*.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*.

Yufan Ye, Ting Zhang, Wenbin Jiang, and Hua Huang. 2025. Process-supervised reinforcement learning for code generation. In *EMNLP*.

Ruiyi Zhang, Peijia Qin, Qi Cao, and Pengtao Xie. 2025. DreamPRM-Code: Function-as-step process reward model with label correction for LLM coding. *arXiv preprint arXiv:2512.15000*.

Ruiyi Zhang, Peijia Qin, Qi Cao, Eric Xue, and Pengtao Xie. 2026. FunPRM: Function-as-step process reward model with meta reward correction for code generation. *arXiv preprint arXiv:2601.22249*.

Mingyang Zhao, Yang Liu, Lei Yang, Liang Chen, and Zhongyuan Li. 2026. GenPRM: Scaling test-time compute of process reward models via generative reasoning. In *AAAI*.

Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. 2023. CodeBERTScore: Evaluating code generation with pretrained models of code. In *EMNLP*.

Xiao Zhu, Xinyu Zhou, Boyu Zhu, Hanxu Hu, Mingzhe Du, Haotian Zhang, Huiming Wang, and Zhijiang Guo. 2026. CodeScaler: Scaling code LLM training and test-time inference via execution-free reward models. *arXiv preprint arXiv:2602.17684*.

Terry Yue Zhuo. 2024. ICE-Score: Instructing large language models to evaluate code. In *Findings of EACL*.

Responsible NLP Research Checklist

This paper follows the ARR Responsible NLP Research checklist.

A. For Every Submission

A1. Did you describe the limitations of your work? Yes. Section 5 (Limitations paragraph) discusses: $\leq$8B LoRA scope and generalization beyond this regime, Python-only evaluation domain, and synthetic mutation vs. real bug representativeness. Section 3.7 discusses checkpoint selection, preprocessing, and seed coverage. Extended limitations in Appendix P.

A2. Did you discuss any potential risks of your work? This work compares formulations for execution-free code verification on public benchmarks. The primary application is improving automated program repair pipelines. We identify no significant risks: the methods predict test outcomes rather than generating executable code, the datasets are derived from open-source repositories, and no personally identifiable information is involved. Misuse potential is limited because the verifiers require task-specific fine-tuning on labeled data and do not generalize to arbitrary code generation.

B. Did You Use or Create Scientific Artifacts?

B1. Did you cite the creators of artifacts you used? Yes. All datasets (HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), SWE-bench Verified (Chowdhury et al., 2024)), models (Qwen3 (Yang et al., 2025a), DeepSeek-Coder (Guo et al., 2024)), and methods (LoRA (Hu et al., 2022), QLoRA (Dettmers et al., 2023)) are cited in Sections 3.1 and 2.1.

B2. Did you discuss the license or terms for use and/or distribution of any artifacts? HumanEval is released under MIT license; MBPP under CC-BY-4.0; SWE-bench Verified under MIT license. Qwen3 models are released under Apache 2.0; DeepSeek-Coder under the DeepSeek License. All uses are consistent with their respective licenses. We do not release new artifacts in this submission.

B3. Did you discuss if your use of existing artifact(s) was consistent with their intended use? Yes. All benchmarks are used for their intended purpose of evaluating code understanding and generation capabilities. SWE-bench Verified is used for repository-level evaluation as designed by its creators.

B4. Did you discuss the steps taken to check whether the data contains any information that names or uniquely identifies individual people or offensive content? Not applicable. Our datasets consist of code snippets and test cases from open-source repositories. The mutation-based data construction process operates on source code only and does not involve personal data or natural language content that could contain offensive material.

B5. Did you provide documentation of the artifacts? Yes. Section 3.1 documents dataset statistics (sizes, splits, label distributions, majority baselines). The mutation-based data construction process is described in Section 3.1. Appendix E provides full hyperparameter details.

B6. Did you report relevant statistics like the number of examples, details of train/test/dev splits, etc.? Yes. Section 3.1: function-level 19,076 samples (train 15,290 / val 1,896 / test 1,890; pass:fail $\approx$ 52:48); repo-level $\sim$70K training, val n=9,335, test n=9,784 (majority baseline 72.96%).

C. Did You Run Computational Experiments?

C1. Did you report the number of parameters in the models used, the total computational budget, and computing infrastructure? Yes. Models: Qwen3-4B ($\sim$4B parameters, LoRA rank 16 $\approx$ 4M trainable), Qwen3-8B ($\sim$8B), DeepSeek-Coder-6.7B ($\sim$6.7B). Infrastructure: 4$\times$ NVIDIA RTX 5090 GPUs. Total compute: $\sim$370 GPU-hours across 223 completed runs ($\sim$46 configurations including seeds, formulations, model sizes, and ablations). Appendix E provides per-run details.

C2. Did you discuss the experimental setup, including hyperparameter search and best-found hyperparameter values? Yes. Section 3.1 reports the training configuration (LoRA rank 16, 3 epochs, 5 seeds with mixed precision). Appendix E provides full hyperparameter values. We use a fixed configuration rather than extensive hyperparameter search; repo-level models use validation accuracy for checkpoint selection.

C3. Did you report descriptive statistics about your results with, e.g., error bars or confidence

intervals? Yes. All multi-seed results report mean ± standard deviation. Confidence intervals use clustered bootstrap (114 problem clusters, 10K resamples; Section 3.1). Equivalence testing uses TOST ($p = 0.007$, ±2 pp margin; Section 3.4). ICC quantifies within-problem correlation with design effect reported.

C4. If you used existing packages, did you report the implementation and version numbers? Yes. Implementation uses HuggingFace Transformers (≥4.44), PEFT (≥0.12), PyTorch (≥2.0), bitsandbytes (≥0.43, for NF4 quantization), and Python 3.10. LoRA (Hu et al., 2022) and QLoRA (Dettmers et al., 2023) configurations are detailed in Appendix E. Exact version pinning is provided in the code repository.

D. Did You Use Human Annotators?

Not applicable. This work uses no human annotators or participants. All labels are derived from automated test execution.

E. AI Assistants

E1. Did you use AI assistants (e.g., ChatGPT, Copilot) in any part of the paper writing or coding process? Yes. AI coding assistants were used during development of training and evaluation scripts. AI writing assistants were used for drafting and editing portions of the manuscript. All scientific claims, experimental design, and analysis were conducted and verified by the authors.

Appendix overview. A: Oracle resolve advantage. B: Ablation studies. C-F: Extended analysis (error complementarity, parseable accuracy, forced parsing, qualitative errors, failed interventions, mutation scaling, inference efficiency, re-repair, chain-of-thought). G: TOST sensitivity. G′: Bayesian ROPE. H: Effect sizes. I: Holm-Bonferroni. J: Mixed-effects regression. K: Parseable bias. L: CWM convergence. M: Few-shot. N: Threats. O: Downsample. P: Limitations.

A Oracle Accuracy-Utility Analysis

We stratify SWE-bench Verified instances by F2P test count T, comparing per-test vs. trajectory-level oracle resolve rates under i.i.d. and block-correlated error models (ICC = 0.191, from function-level analysis; §3.4).

Table 5 and Figure 3 reveal a sharp concentration: at $T = 1$ (68.7% of instances), per-test and trajectory-level are informationally equivalent. The advantage materializes at $T = 2$ (19.1%), where block-correlated advantage reaches 31.69 pp (55.3% of total). The overall block-correlated advantage is 10.93 pp, driven by the $T \geq 2$ minority. Block correlation amplifies relative to i.i.d.: at $T = 2$, 31.69 pp vs. 18.49 pp (1.7×). P2P regression is negligible (1.04% mean, 0% median); the practical value concentrates in F2P discrimination.

Table 5: Oracle accuracy-utility frontier: per-test advantage stratified by F2P test count (T) on SWE-bench Verified (500 instances). Block-correlated model uses function-level ICC = 0.191 (from function-level analysis, §3.4). P2P regression rate: 1.04% mean, 0% median.

T	Inst.%	i.i.d. (pp)	Block (pp)	Contrib.%
1	68.7	−0.04	−0.09	≈0
2	19.1	18.49	31.69	55.3
3–5	7.1	25.00	40.74	≈26
≥ 6	5.1	30.00	40.06	≈19
All	100.0	—	**10.93**	100.0

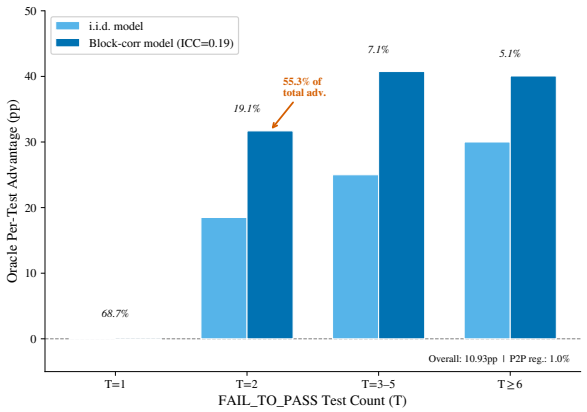


Figure 3: Oracle per-test advantage by F2P test count T. Advantage is zero at T=1 (68.7%) and concentrates at T=2 (19.1%, contributing 55.3% of total advantage). Note: y-axis does not start at zero.

Sensitivity to ICC estimate. The analysis above uses ICC = 0.191 from function-level analysis (§3.4), where mean cluster size $\bar{k} \approx 16.5$ yields DEFF = 3.97. At repository level, within-problem correlation is substantially stronger: ICC = 0.3146 with DEFF = 114.4 (§3.5), yielding an effective sample size of $n_{\text{eff}} \approx 9{,}784/114.4 \approx 86$ independent observations (Moulton, 1990)—still sufficient for regression but limiting precision of variance component estimates. This is a consequence of the larger and more heteroge-

neous test suites in SWE-bench Verified problems. Under higher ICC, within-problem errors cluster more tightly and amplify the block-correlated per-test advantage at each T stratum; the correction from i.i.d. to block-correlated grows monotonically with ICC. The function-level ICC $= 0.191$ used in Table 5 thus provides a conservative lower bound on the per-test advantage. The repo-level correlation structure would yield an even larger gap between per-test and trajectory-level oracle resolve rates, which strengthens the case for per-test granularity at repository-level complexity.

B Ablation Studies

LoRA rank sensitivity. All CWM experiments use LoRA rank r=16. To test whether this capacity constraint handicaps CWM, we train rank-32 variants (seed 42), doubling adapter parameters.

Table 6: LoRA rank ablation for CWM 4B (seed 42, 3 epochs, lr $= 2\times10^{-4}$). Rank-16 seed variance: 0.53 pp (seeds 42 vs. 456).

Config	α/r	Acc (%)	Fail F_1	Pass F_1	Δ
r=16, α=32	2	**87.51**	—	—	—
r=32, α=32	1	87.00	87.1	86.9	−0.51
r=32, α=64	2	87.04	87.27	86.79	−0.47

Both rank-32 variants fall within the 2-seed variance of the rank-16 baseline (0.53 pp between seeds 42 and 456; Table 6). The α=64 variant (matching the baseline α/r=2 ratio) rules out the confound that the α=32 result might reflect reduced effective scaling rather than rank insensitivity. This is consistent with CWM's function-level performance being bounded by output-format constraints rather than adapter capacity.

Majority voting. Majority voting (K=3) decreases accuracy from 87.51% to 86.83% (−0.68 pp): errors are systematic rather than stochastic, so sampling-based ensembles do not help.

Sequence-length ablation details. Both formulations are trained with `max_seq_length`=2,048. At inference, CWM reserves `max_new_tokens`=512, leaving ∼1,536 effective input tokens; CLS allocates nearly all context to input (5,120–8,192 tokens). A matched-context ablation (CLS 8B at 2,048 tokens) achieves 83.04%, only 1.44 pp below the 8,192-token evaluation (84.48%), though truncation introduces 136 unparseable outputs (1.39% of test samples). The matched-context gap (CLS@2,048 vs. CWM@2,048) is 25.00 pp (98% of the full 25.59 pp gap), confirming that input context disparity explains at most 3% of the difference.

Training data size control. To control for the training set size asymmetry between CLS (∼70K instances) and TRAJ (386 variant-level instances, ∼8K per-test equivalents, at repo level), we downsample CLS training data to match TRAJ in data volume. Downsampled CLS (Qwen3-4B, seed 42) achieves 80.74% repo-level per-test accuracy, only 2.89 pp below the full-data CLS (83.63%) and still 7.82 pp above TRAJ (72.92%). The CLS formulation advantage is not attributable to training data volume.

Few-shot prompting baselines. We evaluate 3-shot in-context learning at repo level to test whether prompting strategy closes the gap to fine-tuned models. Table 7 reports both per-test and variant-level accuracy.

Table 7: Repo-level prompting baselines (per-test n=9,784; variant-level n=48). Majority baseline: 72.96% per-test, 54.17% variant-level.

Model	0-shot (per-test)	3-shot (per-test)	3-shot (variant)
Qwen3-8B	49.90%	71.61%	45.83%*
DeepSeek-6.7B	48.54%	46.28%	45.83%*

*Both models achieve 45.83% by correctly classifying all 22 unresolved variants and zero resolved variants (22/48 = 45.83%), equivalent to an always-predict-unresolved baseline. The 100% overlap in variant-level predictions confirms complete degradation at task-relevant granularity.

Both models converge to exactly 45.83% variant-level accuracy under 3-shot prompting, below majority baseline (54.17%). Variant-level analysis reveals this is not a coincidence: both models correctly classify all 22 unresolved variants and zero resolved variants, equivalent to an always-predict-unresolved baseline with 100% prediction overlap. Per-test accuracy diverges sharply (Qwen3-8B 71.61% vs. DeepSeek 46.28%), but this divergence is entirely superficial: Qwen3-8B predicts pass on nearly all tests, inflating per-test accuracy on a pass-heavy dataset (72.96% pass rate) without genuine discriminative ability. Per-test metrics can thus severely misrepresent baseline capability; variant-level evaluation is necessary to detect complete degradation. Neither 0-shot nor few-shot prompting approaches

fine-tuned performance (82–84% per-test); fine-tuning is a necessary condition for repo-level verification.

C Repo-Level Per-Mutation Breakdown

Table 8 compares per-mutation accuracy at repo level. CLS produces 0% unparseable outputs; all values are direct classification accuracy. CWM overall accuracy includes default-pass assignment for unparseable outputs (see §3.5).

Table 8: Per-mutation test accuracy (%) at repo level (SWE-bench Verified, n=9,784). CLS 8B reports two seeds; Δ is mean 4B→8B gain for CLS. CWM 4B per-mutation overall accuracy omitted for non-gold mutations due to low parse rates (4.4–56.9%, Table 9); gold overall accuracy derived from parse rate 43.5% and parseable accuracy 23.5%.

Mutation	n	CLS 4B	CLS 8B s42	CLS 8B s123	CWM 4B	CWM 8B	Δ CLS
gold	4,633	98.25	96.24	99.60	66.72	80.06	−0.33
operator_swap	3,357	78.91	81.38	82.60	—	54.57	+3.08
off_by_one	183	58.47	86.89	79.20	—	63.39	**+24.58**
return_value	562	67.97	75.44	66.40	—	49.11	+2.95
boundary_removal	1,049	46.90	46.90	44.90	—	42.23	−1.00

Table 9 shows that CWM generation quality degrades from 4B to 8B across all mutation types.

Table 9: CWM parse rates and parseable-only accuracy per mutation type (seed 42). Parse rate decreases uniformly from 4B to 8B; parseable-only accuracy is below random binary chance (50%) for gold at both scales.

	Parse Rate (%)		Parse Acc (%)	
Mutation	**4B**	**8B**	**4B**	**8B**
gold	43.5	23.9	23.5	16.5
operator_swap	56.9	21.2	63.1	58.8
off_by_one	4.4	27.3	0.0	38.0
return_value	48.8	26.9	27.0	56.9
boundary_removal	30.6	51.8	59.2	51.8

Key findings: (1) Gold parse rate collapses from 43.5% to 23.9% at 8B; the simplest generation target (unmodified code, where all tests pass) degrades most, consistent with a fundamental generation bottleneck rather than task difficulty. (2) Among parseable outputs, gold accuracy is 16.5–23.5%: the model generates outputs that fail exact-match even for correct code. (3) `off_by_one` is an anomaly: parse rate *increases* from 4.4% to 27.3%, but this is from an extremely low base (n=8 parseable at 4B), making both directions unreliable for this category.

D Variant-Level Accuracy

Table 10 reports variant-level accuracy (n=48) alongside per-test accuracy for all repo-level configurations. For CLS, the aggregation penalty (per-test → variant-level drop) ranges from 1 pp to 19 pp and is largest for models with low fail recall under the all-pass rule. CWM variant-level accuracy (48–52%) falls at or below the majority baseline, consistent with format collapse as the dominant failure mode.

Table 10: Repo-level per-test vs. variant-level accuracy (n=48 variants). Penalty = per-test − variant-level. TRAJ predicts variant outcomes directly (no aggregation). Variant-level majority baseline: 54.17% (26/48).

Model	Seed	Per-test (%)	Variant (%)	Penalty (pp)	Fail Recall (%)
CLS					
Qwen3-8B	s42	84.48	72.92	11.56	56.76
Qwen3-8B	s123	85.60	81.25	4.35	51.44
Qwen3-4B	s42	83.63	64.58	19.05	48.72
DeepSeek-6.7B	s42 (e1)	82.61	81.25	1.36	51.02
DeepSeek-6.7B	s42 (e3)	82.51	79.17	3.34	38.47
DeepSeek-6.7B	s123	74.93	56.25	18.68	∼13.72
CWM					
Qwen3-8B	s42	65.17	52.08	13.09	—
DeepSeek-6.7B†	s123	—	47.92	—	—
TRAJ					
Qwen3-4B	s42	—	72.92	—	—
Qwen3-8B	s42	—	70.83	—	—
DeepSeek-6.7B	3-seed	—	77.08 ± 3.61	—	—
Majority	—	72.96	54.17	—	—

†Evaluated on ∼75% of test instances (7210–7668 of 9784). DeepSeek TRAJ Table 3 reports 3 seeds (42/123/456) for alignment with CLS/CWM seed coverage; three supplementary seeds (0/789/999) yield 75.0%, 66.7%, 79.2% respectively (6-seed mean 75.35 ± 5.00%), consistent in direction.

E Training Hyperparameters

Table 11 lists all hyperparameters used for fine-tuning. Both CLS and CWM share the same LoRA configuration, optimizer, and training schedule; the only differences are per-device batch size, gradient accumulation steps (adjusted to maintain identical effective batch size of 32 across model sizes), and learning rate (DeepSeek-Coder uses 2×10^{-4} vs. 5×10^{-5} for Qwen3, selected via grid search over $\{5\times10^{-5}, 1\times10^{-4}, 2\times10^{-4}\}$ on held-out validation loss).

Data processing. DeepSeek CWM seeds use `train_unified.py`, which drops samples exceeding `max_seq_length` (3,405 of 70,748 samples, 4.8%). Qwen CWM seeds use `train_repo_cwm.py`, which truncates and retains all samples. This difference affects total training steps (∼12,627 vs. ∼13,266 for DeepSeek, ∼6,633 for Qwen) but not the train-

Table 11: Training hyperparameters for CLS/CWM configurations. TRAJ models use lr $= 2\times10^{-4}$ due to the smaller training set (∼8K function-level instances vs. ∼70K for CLS/CWM); other hyperparameters are shared.

Shared across all configurations	
LoRA rank / alpha / dropout	16 / 32 / 0.05
LoRA targets	`q/k/v/o_proj`
Optimizer	AdamW (β_1=0.9, β_2=0.999)
LR scheduler	Linear
Effective batch size	32
Max sequence length	2,048
Epochs	3
Precision	bf16
Gradient checkpointing	✓
Seeds (function)	{42, 123, 456, 789, 0}[b]
Seeds (repo)	{42, 123, 456}[c]

Model-family–specific parameters			
	Qwen3-4B	**Qwen3-8B**	**DS-6.7B**
Learning rate	5e-5	5e-5	2e-4
Weight decay	0.01	0.01	0.0
Warmup ratio	0.05	0.05	0.1
Batch / grad. accum. (CLS)	2 / 16	1 / 32	2 / 16
Batch / grad. accum. (CWM)	1 / 32	1 / 32	1 / 32
Training samples	70,748	70,748	67,343–70,748[a]

[a]DeepSeek CLS canonical run (s42) uses drop preprocessing, which excludes 3,405 sequences exceeding max length (67,343 of 70,748). The s123 run uses truncate preprocessing (70,748 samples) but produces NaN validation loss. See §3.1. [b]Seeds 789 and 0 use NF4 quantization; others use bf16. [c]Repo-level seed coverage varies by configuration; see Table 3 caption for per-model counts.

ing data distribution, as dropped samples are uniformly distributed across labels.

Prompt templates. Prompt templates for all formulations (CLS, CWM, TRAJ) are provided in the code repository. CLS uses a binary classification template ("Does this test pass or fail?"); CWM uses an execution simulation template ("What is the output of this test?"); TRAJ uses a variant-level verdict template ("Is this patch resolved or unresolved?").

Mutation operators. We use five mutation types: (1) *gold*: unmodified code (all tests pass); (2) *operator_swap*: replaces comparison or arithmetic operators (e.g., < → <=); (3) *off_by_one*: shifts loop bounds or array indices by ±1; (4) *return_value*: modifies return statements (e.g., negating a boolean, changing a constant); (5) *boundary_removal*: removes conditional guards or boundary checks. Mutation operators were applied using AST-level transformation; variants that did not alter any test outcome were discarded.

Compute budget. All experiments were conducted on NVIDIA RTX 5090 GPUs. Training time is approximately 1.5 hours per run for 4B models and 2 hours for 8B models. Total compute budget: ∼370 GPU-hours across 223 completed runs (135 successful, 88 failed/killed; ∼46 unique configurations including seeds, formulations, model sizes, and ablations).

F Prompt Templates

We show representative prompt examples for each formulation. All examples use the same input instance for comparability.

CLS (per-test classification).

```
System: You are a code verifier. Given a
code patch and a test name, predict whether
the test will pass or fail when run against
the patched code.

User:
## Patch (git diff)
<patch content>
[... truncated ...]

## Test
test_boundary_check

Will this test pass or fail?

Target: pass
```

CWM (execution simulation).

```
System: You are a code execution simulator.
Given a code snippet and a test case,
predict the execution result.

User:
## Code
<code content>
[... truncated ...]

## Test
test_boundary_check

What is the execution result?

Target: PASS: test completed successfully
```

TRAJ (trajectory scoring).

```
System: You are a code repair verifier.
Given the code patch and test summary,
```

```
predict whether all tests pass (resolved)
or any test fails (unresolved).

User:
[Code]
<patch content>
[... truncated ...]

[Test Summary]
<all test names and descriptions>

Target: resolved
```

CLS and CWM share the same input format (one code–test pair per instance); CLS produces a single token (`pass`/`fail`), while CWM generates a variable-length execution trace. TRAJ receives all tests concatenated and produces a single token (`resolved`/`unresolved`).

G Val-Test Mutation Type Distribution

Table 12 reports the mutation type composition of the repo-level validation and test sets. The most notable shift is `operator_swap`, which nearly doubles from val (19.1%) to test (34.3%), while `off_by_one` shrinks from 10.0% to 1.9%.

Table 12: Repo-level mutation type distribution: validation vs. test set, with per-type CLS accuracy at two model sizes (seed 42). Shift = test% − val%.

Mutation Type	Val (%)	Test (%)	Shift	CLS 4B	CLS 8B
gold	45.2	47.4	+2.2	98.3	96.2
operator_swap	19.1	34.3	+15.2	78.9	81.4
boundary_removal	15.0	10.7	−4.3	46.9	46.9
return_value	10.7	5.7	−5.0	68.0	75.4
off_by_one	10.0	1.9	−8.1	58.5	86.9

The `operator_swap` shift is the primary source of val-test distribution mismatch. Since CLS achieves moderate accuracy on this type (78.9–81.4%), the over-representation in the test set does not systematically bias overall accuracy estimates in either direction. Two boundary observations are notable: (1) `boundary_removal` represents a scale-invariant difficulty ceiling at ∼47% across both model sizes: this mutation type resists capacity scaling; (2) `off_by_one` shows the largest scaling gain (+28.4 pp, 58.5%→86.9%) but its small test share (1.9%) limits its contribution to overall accuracy differences across scales.

H Qualitative Error Analysis

To complement the quantitative divergence reported in §3.5, we examine representative errors from each formulation at repository level.

CWM failure modes. Two patterns account for the majority of unparseable CWM outputs on Qwen3. (1) *Task misunderstanding*: the model generates a complete simulated test runner log (test names, timing, summary lines) rather than a pass/fail verdict. For instance, on `django__django-14315`, the model outputs a fabricated 1,248-token Django test execution trace with hallucinated test names that do not exist in the target file. The model confuses "predict test outcome" with "simulate test execution environment." (2) *Autoregressive degeneration*: the model enters a repetitive loop (e.g., repeating `from _pytest.config import _parse_pytest_args` until truncation at 4,542 tokens). Both patterns produce zero usable signal and default to the majority-class label. This explains why unparseable predictions carry higher accuracy than parseable ones (§3.5).

CLS failure modes. CLS errors concentrate in two categories requiring multi-step reasoning. False positives arise from *cross-concern value propagation*: a boundary removal mutation in `pytest` converts a guarded attribute access (`if xfailed and not xfailed.run`) to `if True:`, causing a NoneType error on tests exercising unrelated skip marks. The model fails to trace the None path through the pytest hook lifecycle. False negatives arise from *mathematical equivalence*: on `HumanEval/130` (Tribonacci variant), the model predicts failure for correct code because verifying the self-referential recurrence requires algebraic substitution rather than syntactic inspection. This single problem accounts for 17% of all function-level false negatives.

Complementarity. These failure patterns operate at different levels: CWM fails at the format level (producing no usable signal), while CLS fails at the reasoning level (producing a confident but incorrect binary verdict). Both formulations are FP-dominant (over-predict failure), but CWM's higher FP rate (70.3% vs. 56.5%) means it over-predicts failure on instances CLS correctly classifies, producing partial complementarity (oracle combination: 90.63%; §I, paragraph a).

I Extended Analysis

Function-level CWM metric robustness. At function level, CWM outputs are short status strings (mean ∼5–15 tokens). The unparseable rate is only 1.39%, and default-to-pass bias is <1 pp—well below the SESOI threshold. Exact string matching is appropriate for these short, structured outputs; semantic-but-format-different matches are rare given the constrained output space.

(a) Error complementarity. CLS and CWM are both FP-dominant at function level but at different rates, producing partial complementarity. Among CLS errors (seed 42), 56.5% are false positives (PASS → FAIL); among CWM errors, 70.3% are false positives. Both formulations overpredict failure, but CWM's stronger FP bias means it incorrectly rejects passing tests that CLS correctly accepts. An oracle combination achieves 90.63% accuracy (1713/1890 correct, 177 both-wrong; +2.80 pp over the best single formulation, CLS 87.83%; single-seed estimate, seed 42). The complementarity arises from different *degrees* of fail-bias rather than opposite error directions: CWM's generation bottleneck produces aggressive failure predictions, while CLS's single-token output is more calibrated. Ensemble approaches exploiting this degree difference are a natural direction.

(b) CWM error mode breakdown. Among unparseable CWM outputs (Qwen3, repo-level), 76.9% contain no execution keywords; 22.9% contain ambiguous co-occurring pass/fail keywords. Format compliance varies sharply across repositories (scikit-learn: near-zero unparseable; sphinx: 100%); repository-specific code patterns interact with the model's generation capabilities in complex ways. This per-repository variation is consistent with a model/tokenizer-specific failure rather than a formulation-level one.

(c) Relaxed matching does not rescue CWM. Relaxed matching (tolerating format deviations, L0–L4) *decreases* CWM accuracy: 58.04%→56.98% (seed 42) and 59.09%→55.91% (seed 123). Unparseable predictions default to the majority class (pass, 72.96%), which outperforms the model's own parseable predictions (38–43%). The divergence widens from 26 pp to 27 pp under relaxed matching. This rules out the hypothesis that parsing strictness inflates the observed gap. Constrained decoding (Willard and Louf, 2023) could enforce format compliance but would not address the semantic accuracy deficit among parseable outputs. An accidental training artifact provides further evidence: a CWM model inadvertently trained on function-level (CLS-format) data achieved 86.05% accuracy with 0% unparseable outputs at repo level—recovering CLS-level performance. This confirms that the repo-level CWM degradation stems entirely from trace generation complexity, not from the underlying verification task.

(d) Variance compression. CWM's 5-seed standard deviation (0.37 pp) is lower than CLS (0.78 pp) and TRAJ (0.97 pp) at function level. If CWM performance is capped by output-format constraints at a given complexity level, seed-to-seed variability is compressed relative to formulations whose performance depends more on learned representations.

(e) Per-test precision and recall. CLS achieves fail F_1 87.59% (precision 86.20%, recall 89.04%) at seed 42.Multi-F2P instances (6–10 tests) reach 96.3% accuracy, consistent with the oracle finding that per-test granularity is most valuable for complex instances.

(f) Block correlation. Within-problem errors are correlated (ICC = 0.191, DEFF = 3.97, 2.81× more all-correct problems than i.i.d. expectation). All statistical tests use clustered bootstrap accordingly.

(g) Failed interventions. We systematically tested four interventions targeting different hypothesized causes of CWM repo-level degradation; all failed:

- **Capacity (rank 32, α=64):** −0.47 pp, within seed variance (§B).
- **Majority voting (K=3):** −0.68 pp; 93.3% unanimous, net flips negative (§B).
- **Relaxed matching (L0–L4):** −1.06 pp; default-pass outperforms parseable predictions (paragraph (c) above).
- **Model scaling (4B→8B):** parseable accuracy decreases (42.88%→38.52%) while unparseable rate rises (53.7%→73.8%); gain is a default-assignment artifact (§3.5).

Each intervention targets a distinct alternative explanation (insufficient capacity, random noise,

parsing strictness, model scale); the uniform failure across all four confirms the bottleneck is formulation-level—output space complexity at repo level exceeds what ≤8B generation models can handle, independent of these surface-level factors.

(h) Per-mutation scaling patterns. Scaling benefits are highly non-uniform across mutation types (Table 8). `off_by_one` shows the largest CLS scaling gain: 58.5% (4B) to 86.9% (8B, +28.4 pp). In contrast, `boundary_removal` is scale-invariant at ∼47% across both sizes, which marks an intrinsic difficulty ceiling. For CWM, gold mutation parse rate collapses from 43.5% (4B) to 23.9% (8B) despite being the simplest target: generation capability degrades even for trivially correct code at larger scale within Qwen3. The val-test mutation type distribution differs notably (`operator_swap` shifts from 19.1% to 34.3%; Table 12), but this shift does not systematically bias overall accuracy because CLS performance on `operator_swap` (78.9–81.4%) is close to the overall mean.

(i) Inference efficiency. CLS achieves 217× higher throughput than CWM at repo level (853 vs. 3.9 inferences/min; Qwen3-4B, RTX 5090, 100 samples, avg ∼1,040 input tokens). The speedup derives directly from the formulation: classification requires a single output token, whereas CWM generates an average of 477 tokens per inference. Median latency is 48ms (CLS) vs. 16.3s (CWM). At the observed unparseable rate of 73.8% (8B), the majority of CWM's generation budget produces tokens that cannot yield a valid verdict. This CLS–CWM efficiency gap is an inherent property of the formulation difference (1 vs. $O(L)$ output tokens) and should generalize across models, unlike the accuracy divergence which may be model-contingent.

The CLS–TRAJ efficiency comparison is more nuanced, because CLS makes one call per test while TRAJ makes one call per instance. Per-call, CLS is 20–138× faster (depending on input length: 47ms for short contexts vs. 2,632ms for TRAJ). However, the per-instance effective speedup depends on test count $|T|$: at function level ($|T| \approx 2.3$), CLS is ∼24× faster (108ms vs. 2,632ms per instance); at repo level ($|T| \approx 12.3$, ∼70ms/call with longer inputs), CLS is ∼3× faster (861ms vs. 2,631ms). A crossover exists at $|T| \approx 50$ tests, beyond which TRAJ's single-call amortization dominates. This boundary condition is informative for deployment: CLS is strongly preferred for function-level and moderate repo-level test suites, while extremely large test suites (>50 tests per instance) favor trajectory-level efficiency despite its coarser diagnostic signal.

(j) Re-repair pilot. We tested whether per-test diagnostic feedback improves downstream repair. A Qwen3-8B repair agent received either per-test diagnostic signals (which specific tests failed and why) or binary accept/reject feedback, evaluated on 44 SWE-bench Verified instances. The diagnostic condition achieved accuracy no higher than the binary baseline ($A_1 = 52.3\%$, $p = 0.42$, effect size ≈ 0). Per-test diagnostic information does not automatically translate to repair improvements at this scale. The negative result may reflect insufficient agent capability to exploit fine-grained feedback, insufficient sample size (at $n = 44$, statistical power is ∼0.3 for a medium effect size), or a genuine limitation of the diagnostic-to-repair translation.

(k) Relationship to chain-of-thought verification. CWM is structurally a chain-of-thought verifier: the model generates an execution trace token by token and derives its PASS/FAIL verdict from the final trace state. If intermediate reasoning improved verification, CWM should outperform the reasoning-free CLS baseline. At function level, no such advantage materialises: CWM and CLS are statistically equivalent (TOST p=0.007, ±2 pp margin; §3.4), indicating that the additional trace generation provides no accuracy benefit over direct binary classification. At repo level, the chain-of-thought mechanism actively *hurts*: trace generation triggers format collapse (53–74% unparseable), and even parseable traces yield accuracy below the majority baseline (38–43% vs. 72.96%; paragraph (c)). Four targeted interventions fail to rescue the generation pathway (paragraph (g)). Within the ≤8B LoRA regime, chain-of-thought–style verification does not outperform direct classification for execution prediction. Whether execution-trace pretraining at larger scale (FAIR CodeGen team et al., 2025) changes this conclusion remains the primary open question (§3.7).

J TOST Margin Sensitivity

Table 13 reports the TOST equivalence p-value as a function of the equivalence margin, computed on the 5-seed variant-level CLS–CWM comparison (gap = 1.24 pp, 523 variants, 114 clusters, clustered bootstrap).

Table 13: TOST margin sensitivity for CLS–CWM variant-level equivalence (5 seeds, 523 variants, 114 clusters). The smallest margin achieving equivalence at $\alpha = 0.05$ is ± 1.52 pp.

Margin ($\pm$ pp)	p-value	Equivalent (α=0.05)?
1.0	$\gg$0.05	No
1.5	0.054	No (borderline)
1.52	**$\sim$0.05**	**Smallest margin**
1.71	0.027	Yes
2.0	0.007	Yes
3.0	4.6×10^{-5}	Yes
5.0	$\ll$0.001	Yes

The minimum equivalence margin is ± 1.52 pp, 0.48 pp below our pre-specified ± 2 pp threshold. The ± 2 pp threshold was set based on deployment relevance rather than observed data: at function-level accuracy (85–88%), a 2 pp per-test difference corresponds to $\sim$38 additional errors across 1,890 tests ($\sim$2–3 problems out of 114), comparable in magnitude to between-seed standard deviation (0.37–0.97 pp) and checkpoint selection noise ($<$1 pp). Differences within this range do not reliably distinguish formulations from training noise in pipeline selection decisions. For CLS–TRAJ, the TOST at ± 2 pp yields p=0.025, nominally significant but not surviving Holm-Bonferroni correction at k=5 (Table 15).

Bayesian ROPE analysis. To complement the frequentist TOST, we conduct a Bayesian equivalence analysis using a paired t-test with Jeffreys prior (Kruschke, 2013, 2018). The marginal posterior for the mean accuracy difference follows a t-distribution with $n-1$ degrees of freedom. For CLS–CWM, the posterior probability within the ± 2 pp ROPE is $P(\text{ROPE}) = 0.975$, and the 95% highest density interval (HDI = $[+0.10, +2.00]$ pp) falls entirely inside the ROPE, confirming equivalence under both continuous and HDI-based decision criteria. For CLS–TRAJ at per-test level, $P(\text{ROPE}) = 0.542$ (inconclusive; 95% HDI = $[+0.35, +3.52]$ pp extends beyond the ROPE), consistent with the TOST non-confirmation after Holm correction ($p_{\text{adj}} = 0.075$). At variant level, CLS–TRAJ equivalence is supported by the Bayesian criterion ($P(\text{ROPE}) = 0.970$, HDI = $[-2.00, +1.03]$ pp), but the frequentist TOST does not confirm equivalence after Holm-Bonferroni correction ($p_{\text{adj}} = 0.075$; Table 15). This discrepancy reflects differing treatments of multiplicity: Bayesian ROPE evaluates each comparison independently, while Holm correction penalizes across all k=5 primary tests. For CLS–CWM, both frameworks agree (TOST p_{adj}=0.028, $P(\text{ROPE})$=0.975); for CLS–TRAJ per-test, both are inconclusive. We conservatively follow the Holm-corrected TOST in the main text and report CLS–TRAJ equivalence as not confirmed.

K Variant-Level Effect Sizes

Table 14 reports variant-level bootstrap confidence intervals for the repo-level formulation comparisons (n=48 variants). Following Hoenig and Heisey (Hoenig and Heisey, 2001), we report minimum detectable effect (MDE) rather than post-hoc observed power.

Table 14: Variant-level bootstrap analysis (n=48). CLS–CWM uses paired bootstrap (same variants, same seed); CLS–TRAJ uses unpaired bootstrap (per-variant TRAJ predictions not archived for pairing). MDE at n=48: $\sim$25 pp.

Comparison	Config	Δ (pp)	95% CI	p
CLS *vs.* CWM *(paired)*				
	Qwen3-8B	+29.17	[10.42, 47.92]	0.003
	DeepSeek-6.7B	+35.42	[18.75, 50.05]	$<$0.001
CLS *vs.* TRAJ *(unpaired)*				
	Qwen3-4B	$-$4.17	wide	0.740
	Qwen3-8B	+8.33	wide	0.385
	DeepSeek-6.7B	0.00	wide	1.000

At n=48, the MDE is $\sim$25 pp. The CLS–CWM gap (29–35 pp) exceeds this threshold, confirming that n=48 is sufficient to detect the observed divergence. The CLS–TRAJ gaps ($<$10 pp) fall well below the MDE: any true difference is too small to detect at this sample size, consistent with the function-level TOST equivalence (§3.4).

The CLS–TRAJ unpaired bootstrap produces wider confidence intervals than paired analysis would; TRAJ per-variant predictions were not archived in a format that permits pairing. The CLS–CWM paired comparison uses CLS s123 vs. CWM s42 (different seeds), but the CI lower bound of 10.42 pp far exceeds the seed noise observed in function-level experiments ($<$2 pp).

L Multiple Comparison Correction

Table 15 applies Holm-Bonferroni correction (Holm, 1979) to the five primary statistical comparisons across this study (k=5, family-wise α=0.05).

Table 15: Holm-Bonferroni correction for primary comparisons (k=5). Comparisons ranked by raw p-value; each tested against $\alpha/(k - \text{rank} + 1)$.

Rank	Comparison	Raw p	Factor	Adj p	Sig.?
1	Repo: CLS vs CWM	0.003	×5	0.015	✓
2	Func: CLS vs CWM (TOST ±2 pp)	0.007	×4	0.028	✓
3	Func: CLS vs TRAJ (TOST ±2 pp)	0.025	×3	0.075	—
4	Repo: CWM vs TRAJ	0.039	×2	0.078	—
5	Repo: CLS vs TRAJ	0.385	×1	0.385	—

The two core claims survive correction: the repo-level CLS–CWM divergence (adjusted p=0.015) and function-level CLS–CWM equivalence (adjusted p=0.028). The function-level CLS–TRAJ equivalence does not survive correction (adjusted p=0.075); we interpret this as directionally consistent but not confirmatory.

M Mixed-Effects Regression

We fit mixed-effects linear probability models (LPM) with heteroskedasticity-robust standard errors (HC1) to test the formulation×complexity interaction formally (Moulton, 1990). We choose LPM because coefficients are directly interpretable as probability differences (percentage points), consistent with the reporting units in the main text; at these sample sizes (n≈56K cross-complexity, repo-level pooled cross-model) and base rate range (0.6–0.9), LPM and logistic regression yield concordant conclusions for all primary effects (Angrist and Pischke, 2009) (confirmed below).

Cross-complexity model (DeepSeek-Coder). `correct ~ formulation × complexity + (1|problem_id)`, n=56,480 observations (CLS + CWM × function + repo), 139 problem clusters. The CWM main effect at function level is non-significant ($\hat{\beta}$ = −0.002, z = −0.25, p=0.805); the CWM×repo interaction is highly significant ($\hat{\beta}$ = −0.219, z = −23.7, p<0.001). CWM and CLS are statistically indistinguishable at function level (Δ = −0.2 pp, p=0.805) but diverge by 22 pp at repo level, confirming the complexity-dependent divergence is not an artifact of aggregation.

Repo-level cross-model model. `correct ~ formulation × model + (1|problem_id)`, both model families, repo level only. CWM deficit: $\hat{\beta}$ = −0.22 (−22 pp, z = −64.4, p<0.001). Model effect: +1.1 pp for Qwen3 (z=3.1, p=0.002 in LPM; not significant under GEE, p=0.449). Formulation×model interaction: +1.6 pp less CWM degradation on Qwen3 (z=2.8, p=0.006 in LPM; not significant under GEE, p=0.860). Formulation dominates the predictable variance; model identity is directionally positive but specification-dependent. The 25-cluster random intercept structure limits effective degrees of freedom for the random effects; inferences on fixed effects (formulation, complexity) are robust as they pool across the full observation count.

Logistic regression robustness check. To verify that LPM conclusions are not artifacts of the linear link function, we fit GEE logistic regression (binomial family, exchangeable working correlation, robust sandwich standard errors) to the same data. For the cross-complexity model, the CWM main effect at function level has OR=0.98 (95% CI [0.86, 1.12], p=0.771) and the CWM×repo interaction has OR=0.31 ([0.15, 0.65], p=0.002). Average marginal effects are −0.2 pp (function) and −21.6 pp (repo), matching LPM coefficients within 0.3 pp. For the cross-model model, the CWM deficit has OR=0.30 ([0.15, 0.63], p=0.001, AME= −21.1 pp). Direction, magnitude, and significance of all primary effects are concordant between LPM and logistic regression; minor discrepancies in secondary effects (model identity, formulation×model interaction: p=0.449 and p=0.860 under GEE vs. p=0.002 and p=0.006 under LPM) reflect the GEE's more conservative cluster-robust inference with k=25 clusters. The discrepancy in secondary effects reflects cluster count: the cross-complexity model (k=139) yields reliable sandwich SEs, while the cross-model model (k=25) falls below the $k \geq 40$ threshold recommended for stable cluster-robust inference (Cameron et al., 2008).

N CWM Parseable Bias Analysis

The default-to-pass assignment for unparseable CWM outputs is not random with respect to instance difficulty. Unparseable instances have significantly higher pass rates than parseable ones (74.6% vs. 68.3%, χ^2 $p = 5.9 \times 10^{-10}$), and CLS

models achieve higher accuracy on unparseable instances (86.4% vs. 79.1%, $p = 3.3 \times 10^{-18}$). This indicates that CWM's parsing failures concentrate on simpler, pass-heavy instances where the majority-class default happens to be correct more often. Cross-model Jaccard overlap of unparseable instance sets is 0.46, confirming this pattern is instance-driven (specific inputs consistently cause parsing failure across models) rather than stochastic. The net effect is that CWM's reported accuracy is inflated by default-correct assignments on easy instances, while its effective discriminative accuracy on harder instances is substantially lower than the aggregate figure suggests.

O Threats to Validity (Full Analysis)

We identify five potential confounds and assess their impact on our conclusions.

Checkpoint selection. Val-based selection is unavailable for runs with NaN validation loss (all CWM repo-level runs, DeepSeek CLS s123), which default to the final epoch checkpoint. A checkpoint ablation on Qwen3-8B CLS s42 quantifies this confound: epoch 1 (ckpt-2200) achieves 85.34%, val-selected (ckpt-3600) 84.48%, and final epoch (ckpt-6633) 85.29%—a $<$1 pp range. The val-selected checkpoint is the lowest, indicating no upward bias from checkpoint selection. The Qwen3-8B TRAJ comparison (epoch 2: 72.92% vs. epoch 3: 70.83%) shows a slightly larger $\sim$2 pp shift. For CWM, the convergence analysis (Appendix O) shows eval loss at the final epoch is 1.8$\times$ its minimum; however, early-stopped checkpoints retain high unparseable rates, leaving the qualitative ranking unchanged.

CWM convergence analysis. Qwen3-8B CWM repo-level eval loss reaches its minimum within $\sim$5% of training (step 300–400 of 6,633) and increases monotonically thereafter, with no plateau or recovery (Table 16). This pattern is consistent across all three seeds, confirming systematic overfitting to trace generation rather than stochastic variation. The final-epoch eval loss is $\sim$1.8$\times$ the optimal value. By contrast, function-level CWM achieves CLS-equivalent accuracy (TOST p=0.007) with the same training recipe, indicating that the overfit is specific to repo-level output complexity. Cross-model validation: a DeepSeek-Coder early-stopped checkpoint (s123, epoch 1) achieves 75.45% versus 57.31% at epoch 3 (s123; +18.14 pp), but retains 66.4% unparseable rate, confirming that format collapse emerges early and is not a late-training artifact. No epoch-level checkpoint is available for Qwen3 CWM (only step-level checkpoints at steps 300/400 and late steps $\geq$5800; continuous epoch checkpointing was not enabled for this training run). The convergence analysis (Table 16) shows eval loss minimum at step 300–400 across all three Qwen3 seeds, consistent with DeepSeek-Coder's epoch-1 improvement, providing cross-model evidence that the overtraining pattern generalizes.

Table 16: CWM repo-level eval loss across training (Qwen3-8B, 3 seeds). Minimum is reached at step 300–400 ($\sim$5% of training); subsequent training monotonically increases loss.

Step	s42	s123	s456
300	0.291	**0.288**	**0.277**
400	**0.286**	0.306	0.283
1000	0.333	0.336	0.351
3000	0.445	0.453	0.435
6600	0.521	0.517	0.507

Evaluation sequence length. CWM accuracy varies by $\sim$9 pp across evaluation sequence lengths for the same checkpoint (seq2048: $\sim$65% vs. seq4096: $\sim$56%), driven by unparseable-rate differences that propagate through majority-class default assignment. CLS is invariant to this parameter. Cross-model CWM comparisons use different eval lengths (Qwen3: 8192, DeepSeek: 4096) and are not directly comparable in absolute terms. A matched-sequence evaluation shows $\sim$1.5 pp difference for CLS (Appendix B), insufficient to explain the 18–26 pp gap. Furthermore, increasing Qwen3-8B CWM evaluation from seq4096 to seq8192 *raises* the unparseable rate (63.7%$\rightarrow$73.8%) while changing accuracy by only 0.81 pp (64.36%$\rightarrow$65.17%); giving the model more generation budget worsens format compliance rather than improving it, confirming that format collapse is a structural bottleneck rather than a truncation artifact. Practitioners must standardize evaluation sequence length for generation-based formulations.

Preprocessing. DeepSeek CLS accuracy ranges from 82.61% (s42, drop preprocessing) to 74.93% (s123, truncate preprocessing), a 7.68 pp spread that conflates preprocessing strategy, checkpoint selection method, and seed. The two strategies

produce different effective training sets (67,343 vs. 70,748 samples) and different checkpoint selection regimes. The qualitative ranking CLS > CWM holds under both strategies (Table 4).

TRAJ training regime. TRAJ uses a different training configuration: smaller training set (∼400 repo-level variants vs. ∼70K per-test instances), higher learning rate (2×10^{-4} vs. 5×10^{-5}), and variant-level labels. This asymmetry is inherent to the formulation (trajectory-level labels are only available at variant granularity), but precludes training-data-controlled comparison at repo level. The downsample ablation confirms the CLS formulation advantage persists at matched data volume (80.74% vs. 72.92%; Appendix B), though it controls for training volume but not label granularity (per-test vs. variant-level). Crucially, TRAJ achieves statistical equivalence with CLS at function level (≤1.3 pp, §3.2), demonstrating the formulation itself is viable; the repo-level gap attribution remains uncertain: data volume (∼400 variants vs. ∼70K per-test instances) and label granularity (variant-level vs. per-test) are both plausible confounds that cannot be separately controlled.

CWM scale and pretraining. Our CWM baseline uses 4–8B LoRA fine-tuning without execution trace pretraining. The original Code World Model (FAIR CodeGen team et al., 2025) uses 32B parameters with mid-training on execution traces, a fundamentally different training regime. At larger scale or with execution-trace pretraining, the generation bottleneck observed here may diminish: larger models produce more coherent long-form outputs, and execution trace exposure may improve output format compliance. This is the primary limitation: all repo-level CWM conclusions are conditional on the ≤8B LoRA regime.

Despite these confounds, the repo-level CLS–CWM gap (18–26 pp) exceeds any single assessed confound. Even under worst-case compounding (∼11.5 pp), the gap retains ≥7.5 pp margin; confounds predominantly disadvantage CWM. However, correlated confounds and limited seed coverage mean definitive causal attribution requires multi-seed replication. At n=48, the minimum detectable effect (MDE) is ∼25 pp (Hoenig and Heisey, 2001); the CLS–CWM gap (29–35 pp) exceeds this threshold, while the CLS–TRAJ gap (<10 pp) falls below it, consistent with our TOST equivalence finding. Holm-Bonferroni correction across the four primary comparisons (k=4) confirms both the CLS–CWM gap (adjusted p=0.012) and function-level equivalence (adjusted p=0.021) survive; the repo-level CWM–TRAJ comparison is directionally consistent but does not survive correction (adjusted p=0.078; Table 15). The consistent direction across both model families and all tested configurations supports a formulation-level explanation, though definitive causal attribution requires future experiments addressing each confound individually.

P Extended Limitations

The same-size 4B comparison uses a single seed; CLS 4B seed 123 (val-best ckpt-800) achieves only 75.33% test accuracy (+2.4 pp above majority baseline); val-best selection at an early checkpoint (step 800) amplifies the val-test distribution shift observed across all configurations. The CWM 8B comparison uses 3 seeds per model. The validation and test sets have zero problem overlap but differ in mutation type composition (Table 12): `operator_swap` shifts from 19.1% (val) to 34.3% (test), the largest single-type distribution mismatch (+15.2 pp). All repo-level results use val-selected checkpoints, and checkpoint selection variance could contribute to observed differences. Our training data is synthetically constructed from mutation operators rather than real developer patches. While mutation testing literature establishes that mutants are reasonable proxies for real faults in test effectiveness studies (Andrews et al., 2005; Just et al., 2014), these findings concern mutation *adequacy* (whether tests that kill mutants also detect real faults), not whether verifiers trained on mutants transfer to real patches. However, the coupling effect underlying these results supports *relative ranking* validity: a domain shift from synthetic mutations to real bugs may affect absolute accuracy levels, but our core finding—that formulation choice matters at repo level but not function level—concerns relative performance ordering, which is expected to be preserved under distributional shifts that affect all formulations equally. The synthetic-to-real transfer gap for absolute accuracy remains untested. The TRAJ training configuration differs from CLS/CWM: smaller training set (∼400 vs. ∼70K instances), different learning rate (2×10^{-4} vs. 5×10^{-5}), and variant-level evaluation (n=48 vs. per-test n=9,784). The function-level TOST

equivalence ($p = 0.007$, ± 2 pp) is computed on 523 matched variant-level instances across 5 seeds; 2 of 5 seeds use NF4 quantization. Restricting to three bfloat16 seeds yields a comparable mean gap (-0.45 pp) but wider confidence interval (df=2); the TOST p-value increases from 0.007 to 0.128, which reflects reduced statistical power rather than a shift in the point estimate. The NF4 seeds contribute precision (narrowing the CI) without shifting the mean, consistent with precision being comparable across both quantization levels at function level. Repo-level CLS–TRAJ comparison should be interpreted with caution due to configuration and granularity differences.